Lecture 4

Top Down and Type Directed Explicit Search

Bottom up search strategies start by constructing small program fragments and then putting together progressively larger fragments until a complete program is constructed. By contrast, Top-down search work by constructing programs with holes, and progressively filling in the details of these holes. This strategy has proven to be particularly effective in constructing functional programs.

Top-down explicit search is also the first context where we encounter another powerful idea in synthesis: the use of types in order to prune the search space. Up to this point, we have been relying on a grammar to define the space of legal programs and have assumed that any legal program with respect to the grammar is valid. Types, however, provide us with an additional mechanism for ruling out invalid programs. Because type systems are generally designed to support local checking, types allow us to rule out invalid program fragments quickly, so we never have to waste time trying different programs.

The idea of leveraging the type system to aggressively prune the search space was proposed almost simultaneously by the team of Osera and Zdancewic [1] and by Feser, Chaudhuri and Dillig [2] at PLDI 2015. In this lecture we focus on the approach of Feser et al., who also expanded on the idea of type directed pruning with some additional pruning strategies based on deductive rules. Before we can explore explore the results of these and other papers, we are going to introduce the simple functional language we will be using for the rest of the section.

4.1 A simple language for list manipulation

In order to explain this algorithm, we are going to be using a simple functional language for manipulating lists. The language will be given by the following grammar:

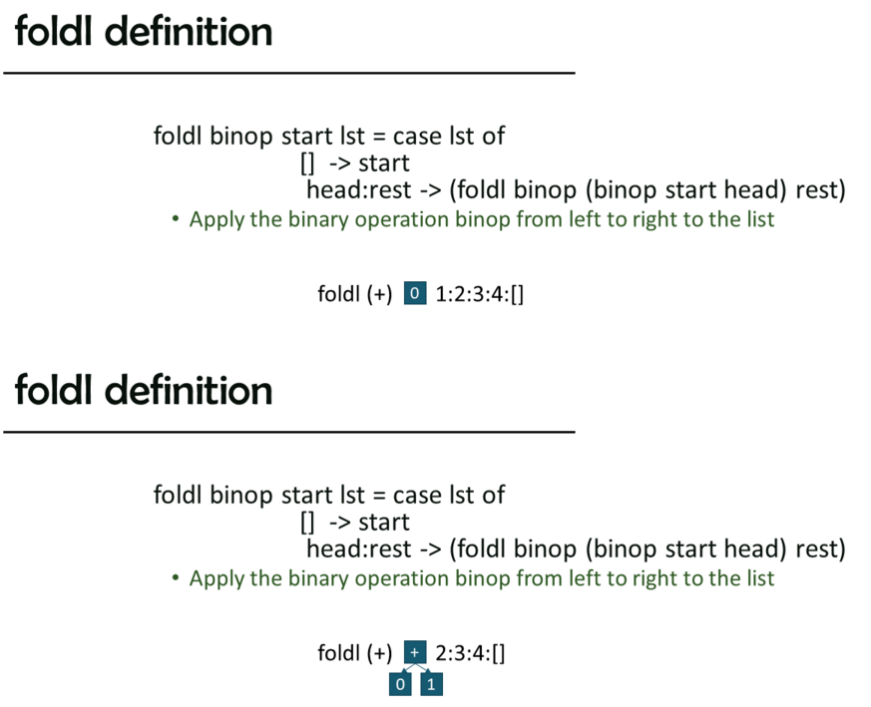
$$
\begin{array}{rcl}
expr & = & var \\
 & | & \lambda x.expr \\
 & | & \text{filter } expr\ expr \\
 & | & \text{map } expr\ expr \\
 & | & \text{foldl } expr\ expr\ expr \\
 & | & boolExpr \mid arithExpr
\end{array}
$$

The symbol var can represent any variable currently in scope. We assume the variable x in the lambda construct is just a fresh variable that does not appear anywhere else in the expression. We assume that the boolean and arithmetic expressions are standard boolean and arithmetic expressions as defined earlier in other languages.

The expressions filter, map and foldl are defined below using functional programming notation.

```
map f lst = case lst of
            []       → []
            head:rest → f(head) : (map f rest)

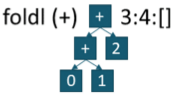
filter p lst = case lst of
            []       → []
            head:rest → if p(head) then head: (filter p rest)
                        else (filter p rest)
foldl binop start lst = case lst of
                      []  → start
                      head:rest → (foldl binop (binop start head) rest)
```



foldl definition

```
foldl binop start lst = case lst of
                  []  -> start
                   head:rest -> (foldl binop (binop start head) rest)
```

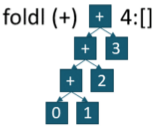
- Apply the binary operation binop from left to right to the list



foldl definition

```
foldl binop start lst = case lst of
                  []  -> start
                   head:rest -> (foldl binop (binop start head) rest)
```

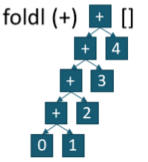
- Apply the binary operation binop from left to right to the list

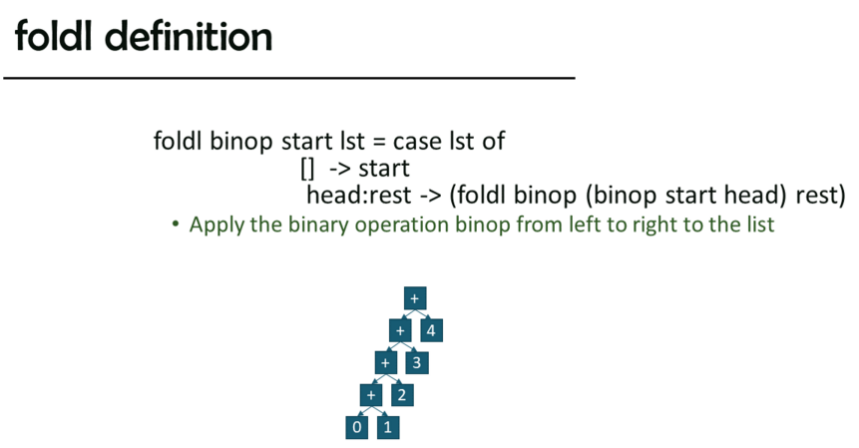


foldl definition

```
foldl binop start lst = case lst of
                  []  -> start
                   head:rest -> (foldl binop (binop start head) rest)
```

- Apply the binary operation binop from left to right to the list





The definitions of map and filter are straightforward. Map applies a given function to all the elements in a given list, and filter creates a new list that excludes any element for which a given predicate evaluates to false.

Fold is a little harder to describe; it essentially applies the binary operation binop from left to right to the list, as illustrated by the animations.

4.2 Running example

To illustrate the main ideas of the algorithm, consider the following example from Feser et al. The goal is to define a function `dropmins` that takes as input a list of lists of integers, where each one of the inner lists corresponds to a list of grades. The output of `dropmins` must be a new list of lists where the lowest grade has been dropped. For example, below is an input/output pair for this function:

```
      Input                 Output
  [ [71, 75, 83],        [ [75, 83],
    [90, 87, 95],          [90, 95],
    [68, 77, 80] ]         [77, 80] ]
```

A valid solution to this problem is shown below.

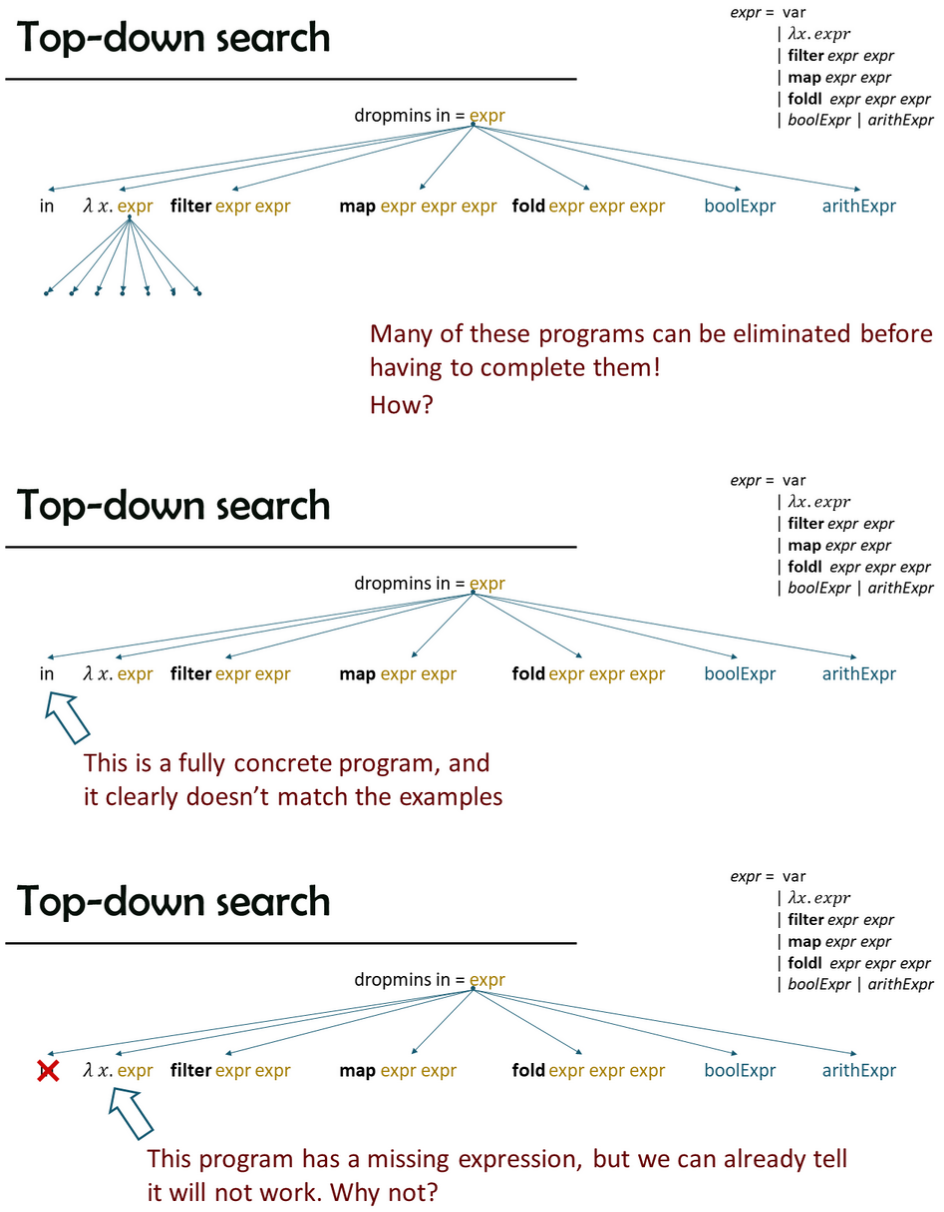
```
dropmins x = map (λ y. filter (λ z. foldl (λ t. λ w. t or (w < z)) False
    y) y) x
```

The same function can be rendered more readable by using where notation.

```
dropmins x = map dropmin x
  where dropmin y = filter isNotMin y
     where isNotMin z = foldl (λ t. λ w. t or (w < z)) False y
```

So the rest of this section will explore how we can go about synthesizing this function.

4.3 Basic top down search



The most basic top-down synthesis algorithm involves simply using the production rules from the grammar in order to generate candidate programs as illustrated in the figure. Every time we produce a concrete program, we can test it against the inputs and check whether it satisfies the constraints. In the figure, we can see that after expanding only one level, the only concrete program

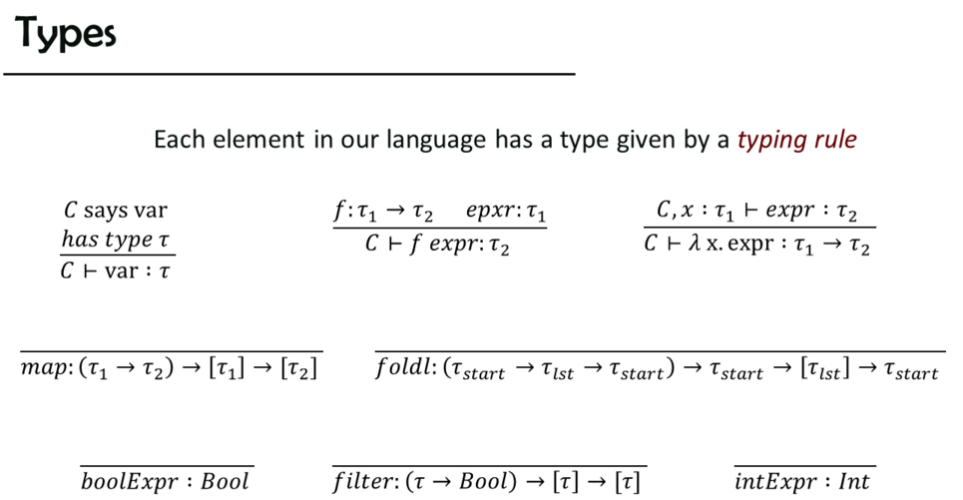
is the program `in`. Evaluating this program, it is clear that it will not produce the desired output, so it can be clearly discarded.

After this first expansion, all the other programs involve un-expanded expressions, so they cannot be evaluated. Even without evaluating them, however, we can already determine that many of them cannot possibly be expanded into a program that works. To understand why, we need to understand more about the type system for this simple language.

4.4 A simple type system

Not all programs in the language above are valid programs. This is because expressions in this language actually have *types*. We have seen languages with types before. For example, the list language from Lecture 2 distinguished between two types: integer and List. Because we only had two types, it made sense to simply distinguish between these in the grammar by separating integer expressions from list expressions. This langauge is different, though, because we have a potentially *infinite* set of types! The reason it is infinite is that we want to support not just integers and lists of integers, but also arbitrarily nested lists of integers, as well as functions.

$$\begin{aligned} \tau &= Int \mid Bool \\ &\mid [\tau] \\ &\mid \tau \rightarrow \tau \end{aligned}$$



The type of an expression is generally given by what is known as a *typing rule*. A typing rule has the form

$$\frac{premises}{Context \vdash expr : \tau}$$

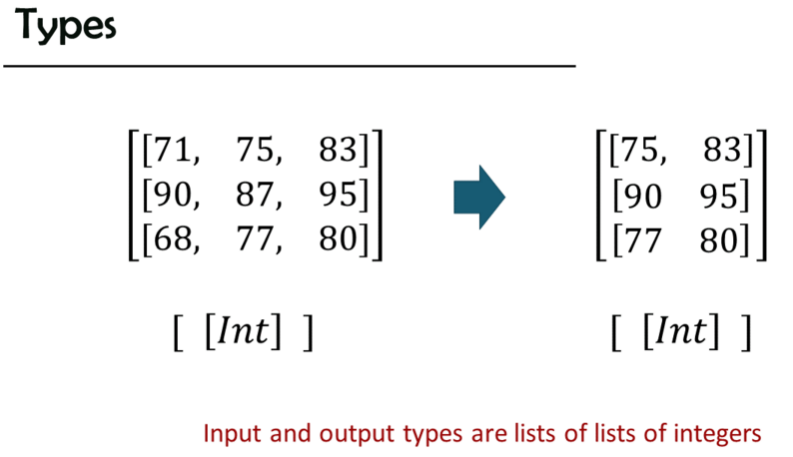
The way to read such a rule is that an expression $expr$ will have type τ in a given $Context$ as long as all the premises are satisfied. The context generally just keeps track of the type of each variable. For example, the rule below

$$\frac{C, x : \tau_1 \vdash expr : \tau_2}{C \vdash \lambda x.expr : \tau_1 \rightarrow \tau_2}$$

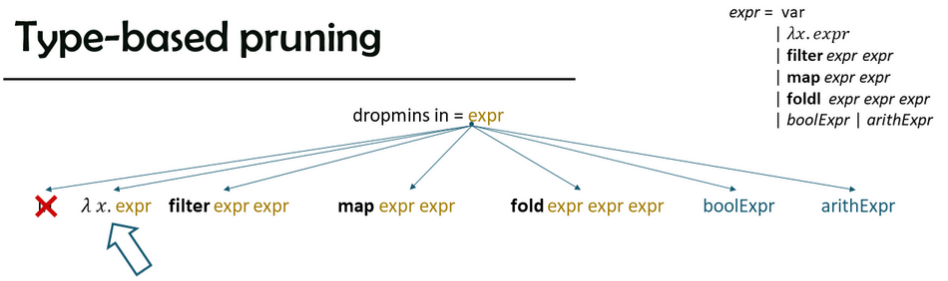
says that an expression $\lambda x.expr$ will have type $\tau_1 \rightarrow \tau_2$ if we can show that the expression $expr$ has type τ_2 in a context that is just like the context C, but that also has x as having type τ_1. The figure shows some additional typing rules.

Note, for example, that according to those rules, fold takes as its first parameter a function that takes in a τ_{start} and then a τ_{lst} and produces a new value of type τ_{start}. The second argument of fold is of type τ_{start}, and the third parameter is a list of values of type τ_{lst}. The result will be a new value of type τ_{start}. Note, for example, that when `foldl` was used in the example earlier, τ_{start} was actually equal to $Bool$ (because the second parameter was the boolean expression `False`), and τ_{lst} was actually equal to Int because the third parameter was a list of integers. The function `(λ t. λ w. t or (w < z))` that was passed as a first parameter therefore had type $\tau_{start} \rightarrow \tau_{lst} \rightarrow \tau_{start} = Bool \rightarrow Int \rightarrow Bool$

4.5 Pruning the search with types



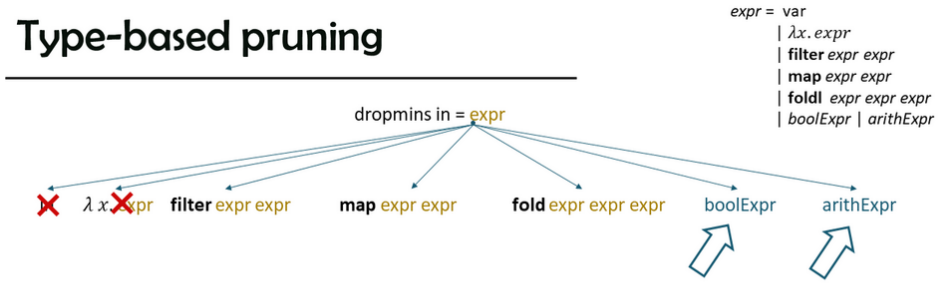
Type-based pruning



$$\frac{expr : \tau_2 \; assuming \; x : \tau_1}{\lambda \, \text{x. expr} : \tau_1 \rightarrow \tau_2}$$

Based on the rule, this expression will have a type $\tau_1 \rightarrow \tau_2$
But we know the output must have type $[\,[Int]\,]$
There is no way those types can be made equal,
so we can discard this expression!

Type-based pruning

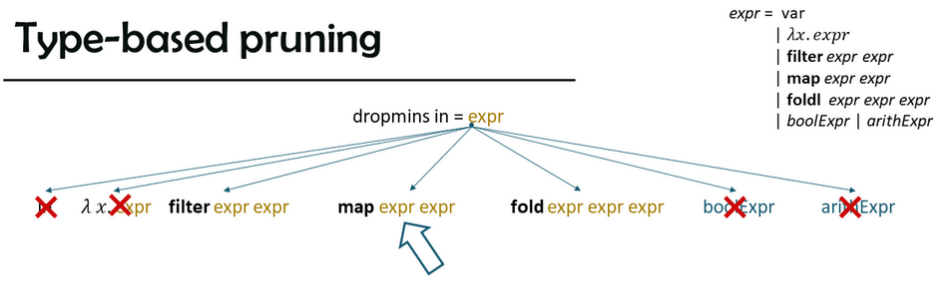


$$\overline{boolExpr : Bool}$$

$$\overline{intExpr : Int}$$

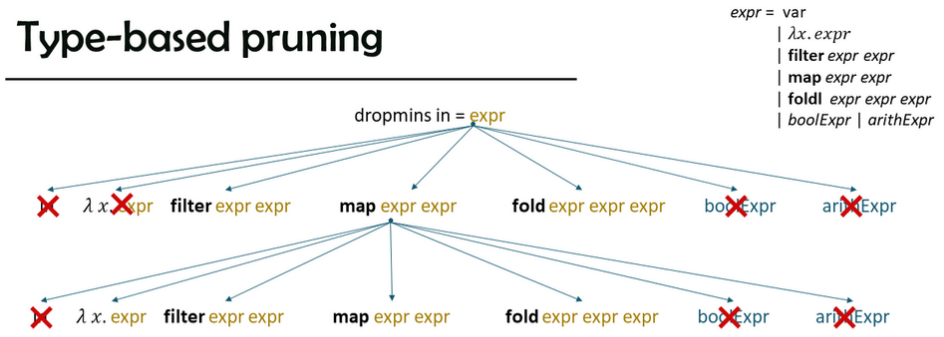
With the same reasoning we can discard both of these expressions
They cannot possibly have the correct type

Type-based pruning



$$\overline{map : (\tau_1 \rightarrow \tau_2) \rightarrow [\tau_1] \rightarrow [\tau_2]}$$

We know the output should be $[\,[Int]\,]$
This means the first expr must be $\tau_1 \rightarrow [Int]$
otherwise the types won't match

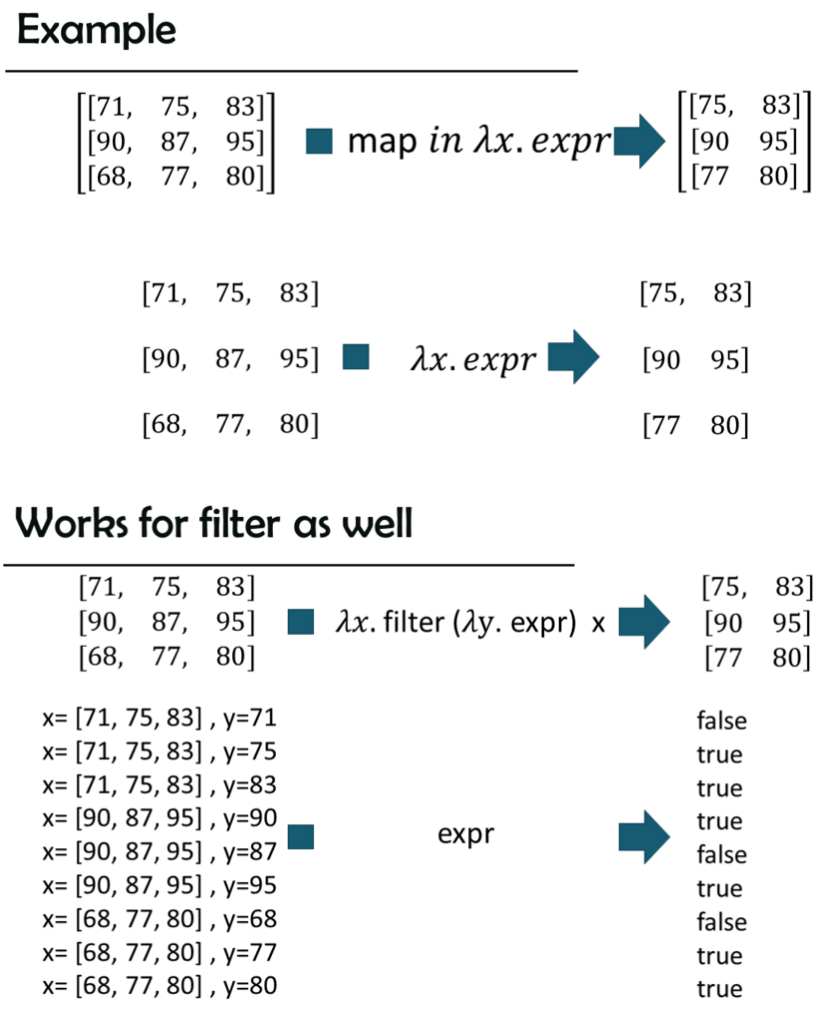


We can quickly dismiss many possible expressions because they cannot produce the type $\tau_1 \rightarrow [Int]$

Now that we know something about the type system for our small language, we can clearly see that many of the expressions from the first expansion can never lead to a correct program and therefore need not be expanded any further. For example, we know that the input and output are both of type $[[Int]]$ because they are both a list of lists of integers. Now, from our typing rules, we know that the expression λ `x. expr` will be of type $\tau_1 \rightarrow \tau_2$. Different expressions $expr$ may lead to different types τ_1 and τ_2, but a lambda must always be a function type. This means that no matter what expression we use for $expr$, $\lambda x.expr$ will never have the type we need for the output, and can therefore be safely discarded. The same is true of the integer and boolean expressions. Regardless of how we instantiate them, they will always have type Int and $Bool$ respectively, so they can never have the desired $[[Int]]$ type, so they can be safely discarded as well.

An expression such as `map expr expr` can have the desired type, but note from the typing rule that in order to have the desired type, the first expression must correspond to a function with type $\tau_1 \rightarrow [Int]$. This imposes some strong constraints on the next level of the search, because we can again rule out any expression that does not have the desired type even before having concrete values for any of its subexpressions.

4.6 Further pruning with deductive rules



We can think of the typing rules as a particular kind of deductive rule that allows us to propagate information about the inputs and outputs down to candidate sub-expressions. One of the key ideas behind the work of Feser et al. is that we can define additional deduction rules for the different constructs in the language in order to prune the search space even more efficiently.

As an example, consider again the sample input/output pair from above, and suppose we are considering a candidate expression `map in λx. expr`. From the definition of map, we can derive input/output rules for the individual expression `expr`, because we know that every element in the input list will be processed by this expression and its output will be added to the output list.

Therefore, when searching for the expression, we can directly check it against its own set of local input/output pairs as illustrated in the figure.

We could further hypothesize that the `λx. expr`, is actually `λx. map x λy. expr`, but even without knowing what `expr` is, we can see that this is not going to work, because the output of map must be of the same length as its input.

On the other hand, if we hypothesize `λx. expr` is actually `λx. filter (λy. expr) x` , we can again propagate the input/output example to the individual expression based on our knowledge of how filter works.

Deduction rules

All of these tricks are codified into deduction rules

Two kinds of rules

- Rules that tell you that a candidate is not going to work
  - for example if input list has different length from output list, map f in will not work
- Rules that tell you how to propagate input/outputs to new expressions

Rule format: when it won't work

$$\frac{conditions}{Examples, expression[f] \vdash f : \bot}$$

Expression involving a function f

Example: rule for map

$$\frac{\mathcal{E} = \bigcup_{1 \le i \le n} \{[a_{i1}, \ldots, a_{ig(i)}] \mapsto [b_{i1}, \ldots, b_{ih(i)}]\} \quad \exists i, j, k, m. \left(\begin{array}{c} 1 \le i \le n \wedge 1 \le j \le n \wedge 1 \le k \le g(i) \wedge \\ 1 \le m \le h(j) \wedge a_{ik} = a_{jm} \wedge b_{ik} \neq b_{jm} \end{array} \right)}{\mathcal{E}, \lambda x.\ \texttt{map}\ f\ x \vdash f : \bot}$$

If the same input value is mapped to multiple output values, then there is no possible function f.

Rule format: when we can propagate

$$\frac{conditions}{Examples, expression[f] \vdash f : New\ Examples}$$

Example rules for map

$$\frac{\begin{array}{c} \mathcal{E} = \bigcup_{1 \le i \le n} \{[a_{i1}, \ldots, a_{ig(i)}] \mapsto [b_{i1}, \ldots, b_{ig(i)}]\} \\ \mathcal{E}' = \bigcup_{1 \le i \le n} \{a_{i1} \mapsto b_{i1}, \ldots a_{ig(i)} \mapsto b_{ig(i)}\} \\ ((a, b) \in \mathcal{E}' \wedge (a', b') \in \mathcal{E}' \wedge a = a') \Rightarrow b = b' \end{array}}{\mathcal{E}, \lambda x.\ \texttt{map}\ f\ x \vdash f : \mathcal{E}'}$$

This intuition about how to propagate inputs, or how to tell when a candidate is not going to work can be formalized into deductive rules. The rules describe how given a candidate expression involving an unknown subexpression f and a set of input/output examples, one can either propagate the input/output examples down to the unknown sub-expression or alternatively establish that no this is a dead-end for the search. The figure shows to examples of rules

corresponding to two of the cases described above. One is the rule that explains how a set of examples for an expression involving map can be propagated to the sub-expression. Note that the rule as many conditions, for example, the rule requires that the examples are not contradictory, requiring different outputs for the same input. Another rule in the figure is the rule for the case when the inputs and outputs are of different length. In that case, the rule tells us that this is a dead end, as no function can satisfy that constraint.

References

[1] Peter-Michael Osera and Steve Zdancewic. Type-and-example-directed program synthesis. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '15, pages 619–630, New York, NY, USA, 2015. ACM. ISBN 978-1-4503-3468-6. doi: 10.1145/2737924.2738007. URL http://doi.acm.org/10.1145/2737924.2738007.

[2] John K. Feser, Swarat Chaudhuri, and Isil Dillig. Synthesizing data structure transformations from input-output examples. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '15, pages 229–239, New York, NY, USA, 2015. ACM. ISBN 978-1-4503-3468-6. doi: 10.1145/2737924.2737977. URL http://doi.acm.org/10.1145/2737924.2737977.